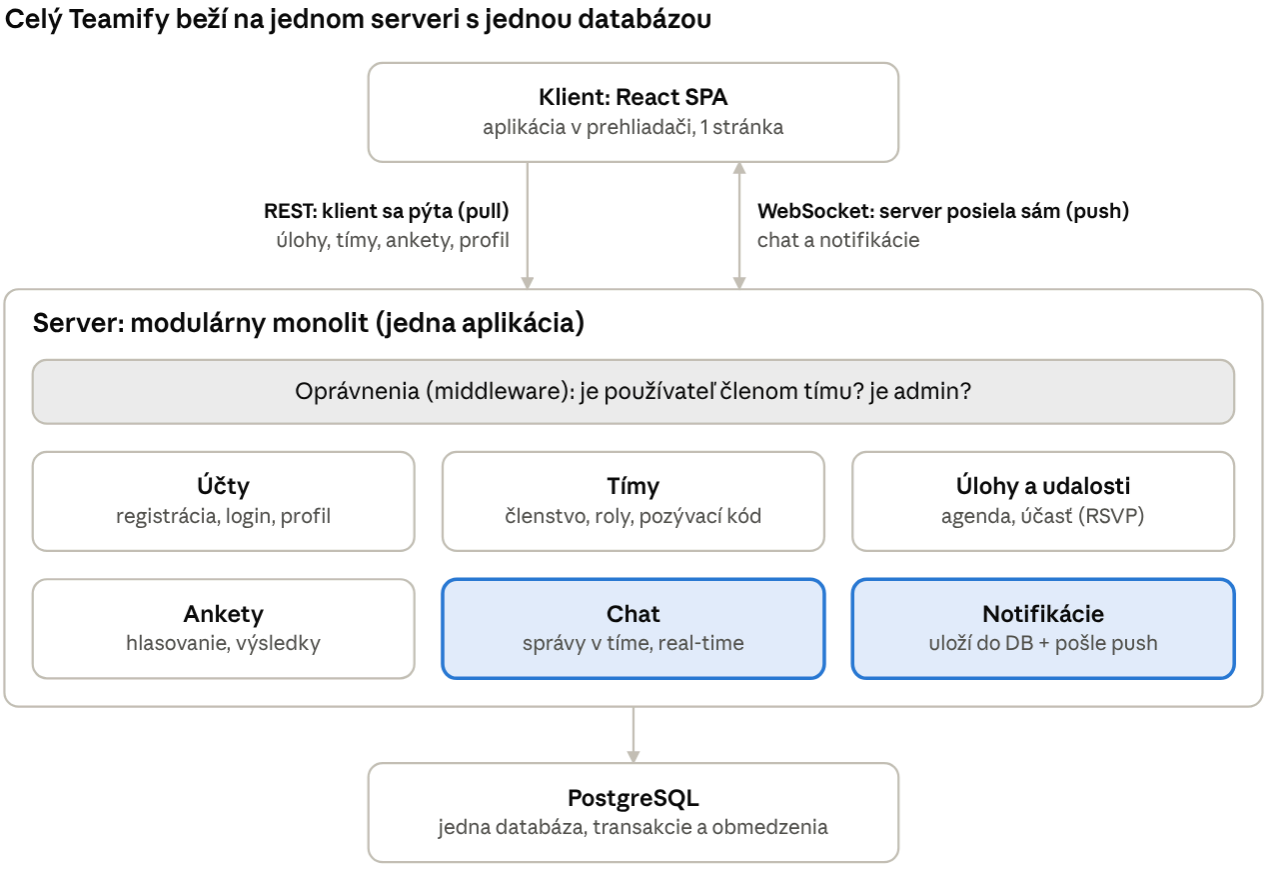
ADR-001: Architektúra systému Teamify

Oct 29, 2026 · @Ogi

Teamify implementujeme ako modulárny monolit nad jednou databázou PostgreSQL, s REST API a WebSocketmi na tom istom serveri.



architektúra Teamify · klient, server so 6 modulmi, databáza

Zvýraznené moduly posielajú správy cez WebSocket.

Pole	Hodnota
Stav	Navrhnuté (po schválení tímom zmeniť na Prijaté)
Dátum	29. 10. 2026
Rozhodovatelia	Lukáš Ogurčák a projektový tím
Rozsah	Celková architektúra backendu a komunikácie klient – server
Podklady	Teamify.txt (31 user stories), diskusia s Claude, Teamify – poznámky k architektúre

Kontext

Teamify je univerzitné zadanie: aplikácia na osobné úlohy a tímovú spoluprácu (31 user stories).

Architektúru najviac ovplyvňujú tri veci:

- **Real-time:** chat a notifikácie musí server poslať sám, bez otázky klienta.
- **Roly v tíme:** člen alebo admin, menia sa za behu; tím nesmie ostať bez admina.
- **Kaskády:** zmazanie tímu alebo účtu zasiahne udalosti, ankety aj chat.

Obmedzenia: malý tím bez skúseností s distribuovanými systémami, obmedzený čas semestra a hodnotenie návrhu architektúry vyučujúcim. Offline režim zadanie nevyžaduje.

Rozhodnutie

Variant A: jedna aplikácia rozdelená na 6 modulov nad jednou databázou PostgreSQL (schéma hore).

- **Klient:** webová aplikácia v Reacte typu SPA; bežné operácie cez REST, chat a notifikácie cez WebSocket.
- **Oprávnenia:** jedno centrálne miesto (middleware) pre všetky požiadavky.
- **Stack:** čo tím ovláda, napr. Spring Boot, NestJS alebo Django + PostgreSQL + React.
- **Zjednodušenia:** jeden server bez Redisu; notifikácia sa vždy uloží do DB aj pošle cez WebSocket; do tímu sa vstupuje pozývacím kódom.

Zvažované možnosti

Porovnali sme tri principiálne odlišné architektúry; A vyhráva jednoduchosťou a konzistenciou dát, čo pre tento rozsah prevažuje nad škálovaním.

Kritérium	A: Modulárny monolit	B: Mikroslužby s event busom	C: Serverless / BaaS
Popis	1 aplikácia, moduly, 1 PostgreSQL, REST + WebSocket	samostatné služby s vlastnými DB, API gateway, broker (RabbitMQ/Kafka)	Firebase alebo Supabase, cloudové funkcie, pravidlá v DB
Rýchlosť vývoja	stredná	nízka	vysoká
Škálovanie	celok naraz	po službách	automatické

Kritérium	A: Modulárny monolit	B: Mikroslužby s event busom	C: Serverless / BaaS
Konzistencia dát	transakcie	eventual consistency	závisí od platformy
Real-time	WebSocket na serveri	notifikačná služba + broker	vstavané odbery zmien
Prevádzková náročnosť	nízka	vysoká	takmer nulová
Najväčší pain	správa WebSocket spojení	dual write (outbox pattern)	posledný admin a kaskády v security rules
Výsledok	**zvolené**	zamietnuté	zamietnuté (záložná voľba)

B zamietnuté: viac času by išlo do infraštruktúry než do funkcií a pribudli by ťažké problémy (outbox pattern, sagy, zastarané roly v JWT tokenoch). Zmysel má, len ak predmet vyžaduje distribuovaný systém.

C zamietnuté: vendor lock-in, logika v ťažko testovateľných pravidlách databázy a pravidlo posledného admina sa v nich nedá rozumne napísať. Vyučujúci môže namietať, že architektúra je prevzatá, nie navrhnutá.

Dôsledky

Získame	Platíme
jeden celok, ľahké spustenie, testovanie a ladenie	škáluje sa len celá aplikácia naraz
zmazanie tímu aj so všetkým v jednej transakcii	pri 2+ serveroch treba doplniť Redis pub/sub
žiadny outbox, sagy ani zastarané tokeny	hranice modulov treba strážiť v code review
posledného admina a 1 hlas stráži databáza	real-time vrstvu píšeme sami

Riziká a mitigácie

Najťažšie nebudú CRUD operácie, ale real-time doručovanie, súbežnosť a invarianty; každé riziko má známe riešenie.

Riziko	Prejav	Mitigácia
Správa WebSocket spojení	používateľ má viac zariadení, spojenie padá, správy sa strácajú	mapa používateľ → spojenia, autentifikácia tokenom pri nadviazaní, klient si pamätá ID poslednej správy a po reconnecte dotiahne zmeškané cez REST
Race condition pri adminoch	dvaja admini si naraz odoberú práva, tím ostane bez admina	kontrola „ostane aspoň 1 admin“ v transakcii so zamknutím riadkov (SELECT … FOR UPDATE)
Zabudnutá kontrola oprávnení	endpoint bez overenia roly = bezpečnostná diera	centrálny middleware/policy, testy oprávnení pre každý endpoint
Dvojité hlasovanie	súbežné hlasy toho istého používateľa	unikátne obmedzenie (anketa, používateľ) v DB
Zmazanie účtu	diery v histórii chatu	správy nemazať, autora nahradiť textom „Zmazaný používateľ“
Časové pásma v agende	posunuté udalosti medzi členmi	ukladať v UTC, na miestny čas prevádzať až v klientovi
Rozmazané hranice modulov	moduly čítajú cudzie tabuľky	prístup len cez rozhrania modulov, kontrola v code review

Otvorené otázky a prehodnotenie

Zadanie nechává tri veci nejasné; treba ich potvrdiť s vyučujúcim pred návrhom DB schémy.

- [ ] Pripojenie k tímu: pozývací kód (náš predpoklad), pozvánka alebo žiadosť o vstup?
- [ ] Posledný admin: môže tím opustiť, ak nevymenuje nástupcu, alebo sa tím zmaže?
- [ ] Zmazanie účtu: čo s jeho udalosťami, hlasmi v anketách a tímami, kde je jediným adminom?
- [ ] Schválenie ADR tímom a zmena stavu na Prijaté.

Rozhodnutie prehodnotíme, ak:

- predmet bude výslovne vyžadovať distribuovaný systém → variant B;
- predmet nebude hodnotiť návrh backendu a prioritou bude rýchlo funkčná aplikácia → variant C cez Supabase (nie Firestore);
- bude treba viac inštancií servera → doplniť Redis pub/sub, architektúra ostáva.

Ďalšie kroky: diagram modulov monolitu a ich prepojení, návrh databázovej schémy.